Ficha: 3406012

Arquitectura Cliente Servidor

Johan Ospina Fontecha
Bruce Derek Peña Contreras

¿Qué es la arquitectura MVC?

La arquitectura cliente-servidor es un modelo de computación distribuida donde las tareas se dividen entre los clientes, que solicitan servicios mediante interfaces (frontend), y los servidores, que los proveen administrando la lógica y los datos (backend) a través de una red (Arsys, s.f.; Daemon4, s.f.; Supermicro, s.f.). Su propósito principal es aplicar la separación de responsabilidades en el software para centralizar la seguridad y los datos, permitir la escalabilidad independiente de ambos componentes y optimizar el rendimiento de dispositivos ligeros al delegar el procesamiento pesado al servidor (Arsys, s.f.; Daemon4, s.f.).

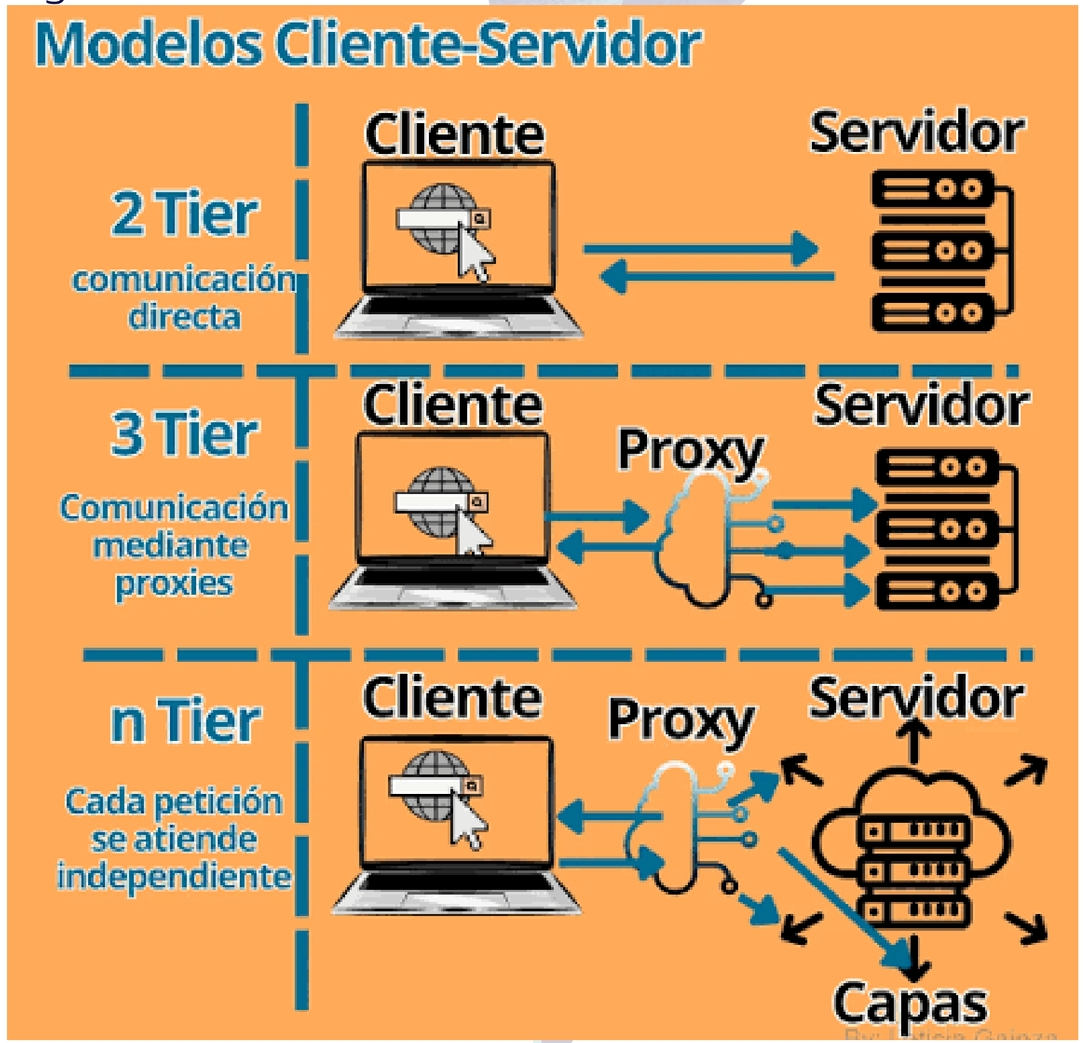
Figura 1. *Arquitectura cliente servidor.*

Edi Vieito (2026)

¿Cómo funciona la arquitectura cliente-servidor?

En cuanto a su funcionamiento, este modelo se basa en un flujo bidireccional de solicitud y respuesta (request-response) soportado por protocolos de red como TCP/IP, HTTP o FTP (Arsys, s.f.; Daemon4, s.f.). El proceso inicia cuando el cliente envía una petición formal a través de la red (Arsys, s.f.; Daemon4, s.f.). A continuación, el servidor recibe el mensaje, valida la solicitud, ejecuta la lógica de negocio requerida y realiza las consultas pertinentes en la base de datos (Arsys, s.f.; Daemon4, s.f.). Por último, el servidor emite una respuesta con los datos procesados o el estado de la operación hacia el cliente, el cual interpreta la información y la presenta visualmente al usuario (Arsys, s.f.; Daemon4, s.f.).

Figura 2. *Modelos cliente servidor.*



Compilador QA (s.f.)

Solicitud

Generación y envío de la solicitud (Petición): El proceso inicia en el cliente cuando el usuario realiza una acción en la interfaz que requiere datos o servicios externos (Arsys, s.f.; Daemon4, s.f.). La aplicación cliente estructura esta demanda en forma de paquete y la envía a través de la red utilizando protocolos de transporte como TCP/IP y de aplicación como HTTP, HTTPS o FTP (Daemon4, s.f.).

Procesamiento

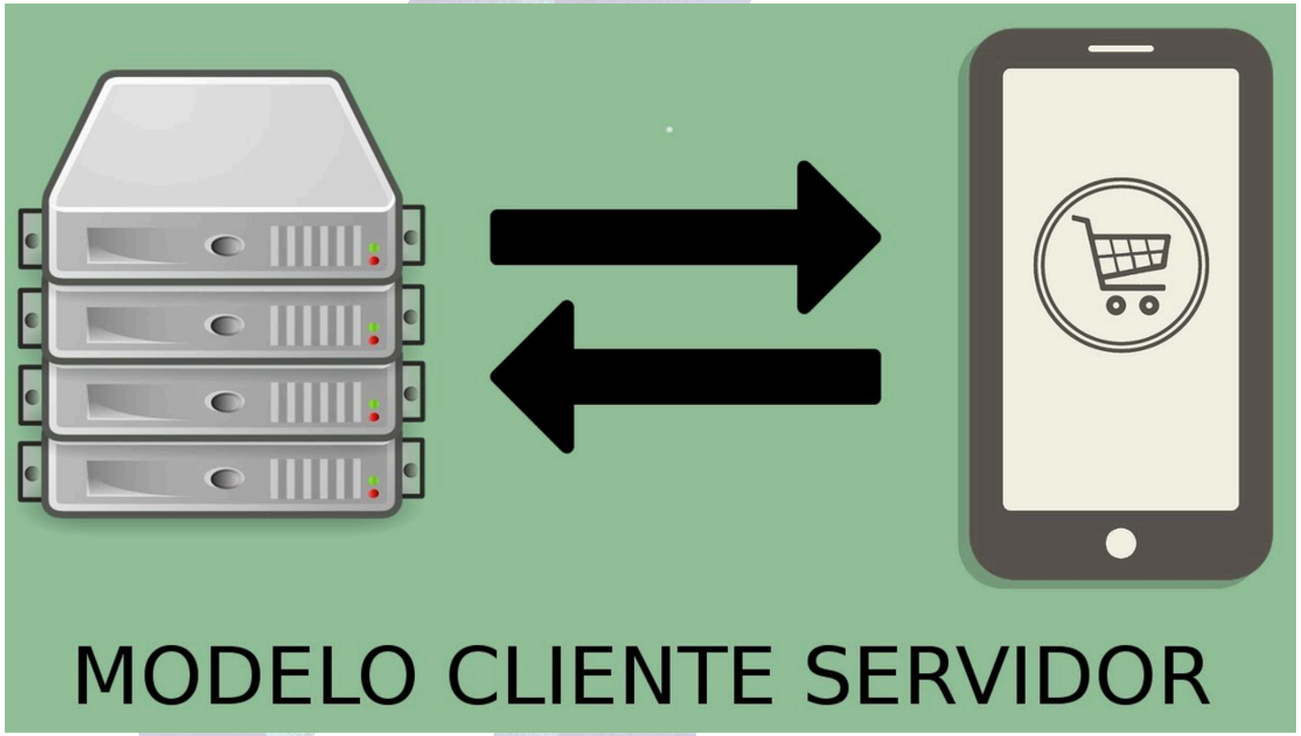
Recepción y procesamiento (Gestión del servidor): El servidor recibe el mensaje enviado por la red y actúa como el motor central de cómputo (Arsys, s.f.). En esta fase, el servidor analiza la petición, valida la autenticidad o permisos del cliente, ejecuta las reglas de negocio pertinentes y consulta o actualiza las bases de datos y archivos centralizados (Arsys, s.f.; Daemon4, s.f.).

Salida

Devolución e interpretación (Respuesta): Una vez obtenido el resultado, el servidor empaqueta la información –ya sea en forma de datos estructurados (JSON/XML), páginas web (HTML) o códigos de estado– y la retransmite al cliente a través de la red (Arsys, s.f.). Finalmente, el cliente recibe dicho paquete, interpreta el contenido y actualiza su interfaz visual para mostrar el resultado al usuario (Arsys, s.f.; Daemon4, s.f.).

Componentes

Figura 3. Cliente y Servidor.



Fazt (2017)

- **Cliente:** Es la aplicación o dispositivo que actúa como interfaz de usuario (frontend) y desencadena la interacción enviando solicitudes de servicio hacia el servidor (Arsys, s.f.; Daemon4, s.f.). Puede tomar la forma de un navegador web, una aplicación móvil o un software de escritorio especializado (Arsys, s.f.; Daemon4, s.f.).
- **Servidor:** Es el sistema central (backend) que permanece a la espera de las solicitudes de los clientes para procesarlas y devolver un resultado (Arsys, s.f.; Daemon4, s.f.). Alberga la lógica de negocio, ejecuta el procesamiento de cómputo y gestiona los recursos centralizados como bases de datos o archivos (Arsys, s.f.; Daemon4, s.f.; Supermicro, s.f.).

Componentes

Figura 4. Https.

Equipo de Keyfactor (2022)

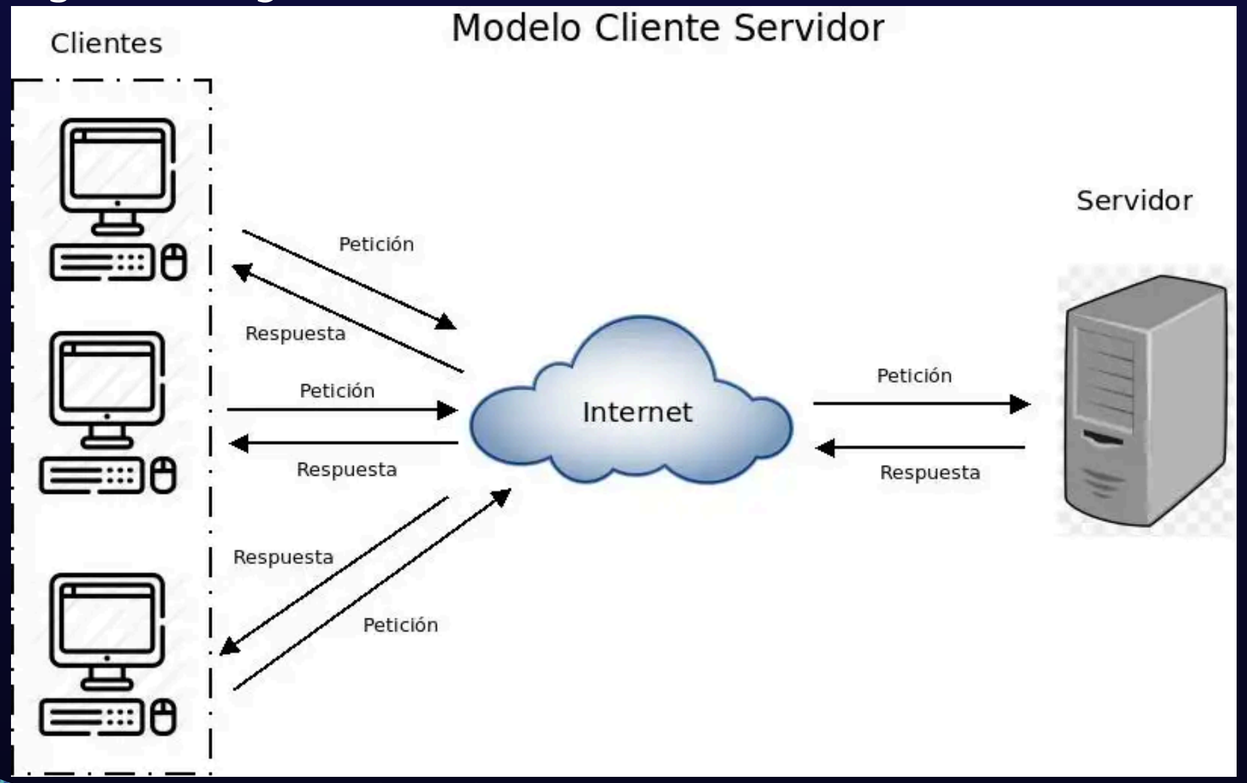
- **Red:** Es la infraestructura de comunicaciones (como Internet o una red de área local LAN) que proporciona el medio de transmisión físico y lógico para el intercambio de datos entre los clientes y los servidores (Arsys, s.f.; Daemon4, s.f.).
- **Protocolos (como HTTPS):** Son los conjuntos de reglas estandarizadas que determinan el formato, la sintaxis y el control de la información transmitida entre el cliente y el servidor (Arsys, s.f.; Daemon4, s.f.). Mientras que TCP/IP administra la entrega correcta de paquetes a través de la red, protocolos de aplicación como HTTP o HTTPS (Hypertext Transfer Protocol Secure) definen la estructura de las peticiones web, añadiendo en el caso de HTTPS una capa de cifrado para garantizar la seguridad e integridad de la información sensible (Arsys, s.f.; Daemon4, s.f.).

Problemas que resuelve

La arquitectura cliente-servidor resuelve el problema de la gestión dispersa e ineficiente de recursos en sistemas donde múltiples usuarios necesitan acceder a la misma información o servicios (Daemon4, s.f.; Supermicro, s.f.).
Necesidad que resuelve

- **Inconsistencia y duplicación de datos:** Resuelve la dispersión de la información al concentrar la base de datos en un punto central, garantizando una única fuente de verdad actualizada en tiempo real para todos los usuarios (Daemon4, s.f.).
- **Limitación de hardware en clientes:** Elimina la necesidad de que los dispositivos de los usuarios dispongan de gran capacidad de almacenamiento o procesamiento, delegando las operaciones complejas al servidor (backend) (Arsys, s.f.).
- **Vulnerabilidad en la seguridad:** Resuelve los riesgos de exposición al centralizar la autenticación, las políticas de acceso y la protección mediante cortafuegos y cifrado en el servidor (Daemon4, s.f.).
- **Acoplamiento rígido del software:** Resuelve la dificultad de actualizar sistemas monolíticos mediante la separación de responsabilidades, permitiendo modificar la interfaz (frontend) o la lógica interna de forma independiente (Arsys, s.f.; Daemon4, s.f.).

Figura 5. *Diagrama cliente servidor.*



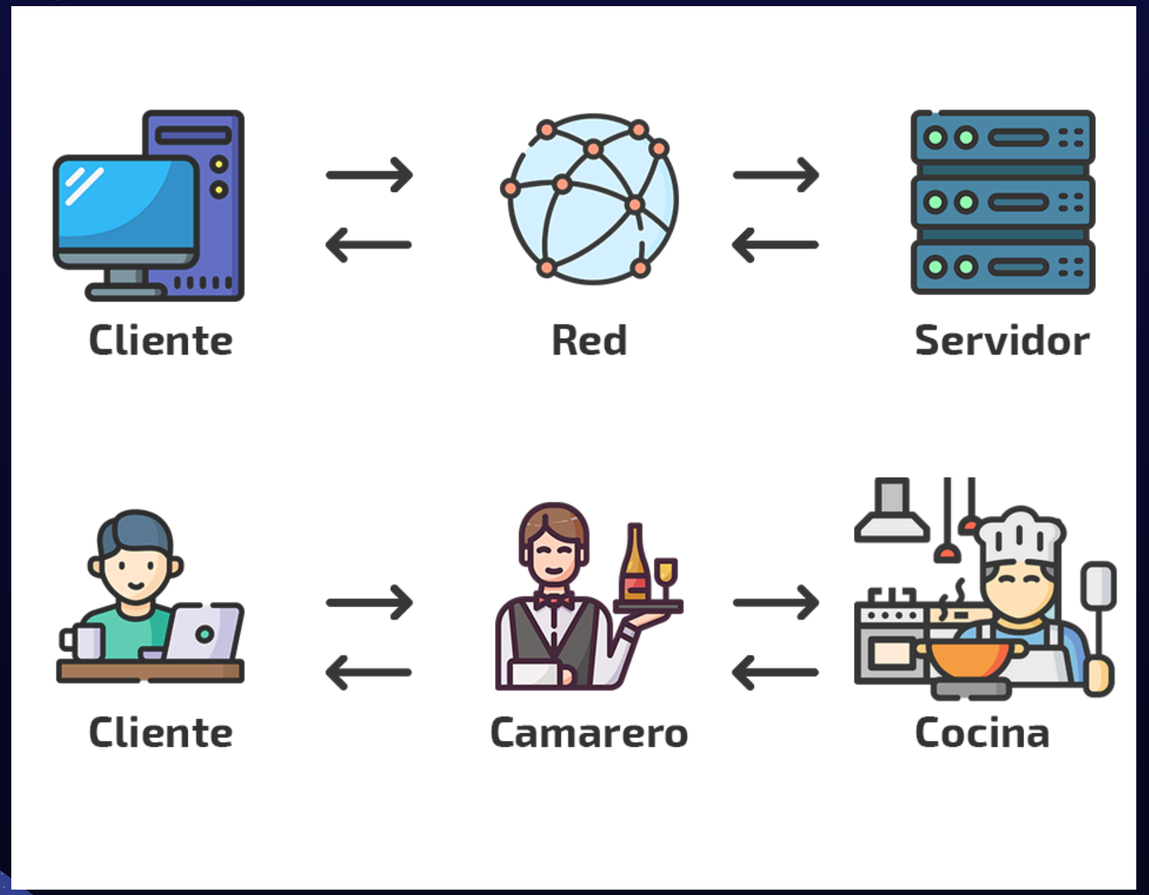
Andrés Schiaffarino (2019)

Casos de uso y conveniencia

El uso de la arquitectura cliente-servidor es aconsejable en los siguientes escenarios:

- **Acceso concurrente y multiusuario:** Cuando múltiples clientes requieren consultar o actualizar información compartida de manera simultánea, como en tiendas en línea, sistemas bancarios o plataformas de gestión empresarial (Arsys, s.f.; Daemon4, s.f.).
- **Manejo de información sensible:** Cuando el sistema gestiona datos críticos que exigen controles estrictos de seguridad, auditorías centralizadas y copias de seguridad continuas (Daemon4, s.f.).
- **Despliegue en dispositivos heterogéneos:** Cuando los usuarios acceden a la aplicación desde diferentes plataformas (navegadores web, teléfonos móviles o programas de escritorio) que dependen de un núcleo de datos común (Arsys, s.f.; Daemon4, s.f.).
- **Proyectos en crecimiento o escalables:** Cuando se prevé un incremento en el volumen de usuarios o datos, ya que permite escalar la infraestructura del servidor (incrementando memoria, procesamiento o añadiendo más nodos) sin alterar las aplicaciones cliente (Arsys, s.f.; Daemon4, s.f.).

Figura 6. *Flujo cliente servidor.*



Edi Vieito (2026)

Ejemplo

El ejemplo demuestra la arquitectura cliente-servidor en Python mediante sockets TCP/IPv4 bajo el ciclo solicitud-respuesta. El script servidor.py vincula la dirección localhost:8000 (bind) y entra en escucha activa (listen); al recibir una conexión, la función accept() asigna un canal dedicado para recibir la petición del cliente (recv), decodificando los bytes recibidos de UTF-8 a texto. Por su parte, cliente.py establece la conexión (connect), serializa una solicitud HTTP de texto a bytes (encode) y la transmite por la red (sendall). Finalmente, el servidor genera una respuesta HTTP estructurada con cuerpo JSON, la devuelve en bytes hacia el cliente y libera la sesión (close), demostrando de forma directa cómo TCP garantiza el transporte físico de la información mientras HTTP estructura el mensaje que viaja en ella.

Ejemplo

MONOLITICA

Arquitectura monolitica

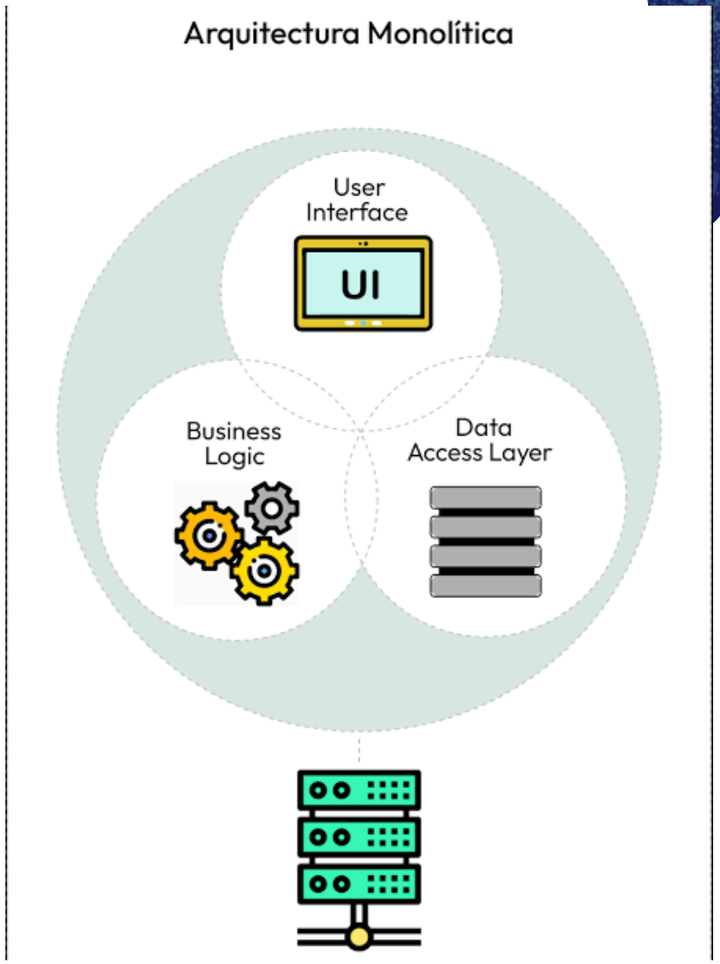
¿Que es?

La arquitectura monolítica es un modelo de desarrollo de software en el que los principales componentes de una aplicación se encuentran integrados y funcionan como una sola unidad. Esto significa que funcionalidades como el inicio de sesión, la gestión de usuarios, los productos, los pagos y los pedidos forman parte de una misma aplicación.

en este modelo la interfaz de usuario, la lógica del negocio y el acceso a los datos se encuentran relacionados y se despliegan como una sola unidad de software.

(OpenStax, 2024).

Figura 7. Arquitectura monolitica



https://www.esic.edu/rethink/tecnologia/wordpress-que-es-para-que-sirve-desarrollo-web-c

Ejemplo:

Figura 8. Gestión de contenidos y sistemas de publicación web

https://www.esic.edu/rethink/tecnologia/wordpress-que-es-para-que-sirve-desarrollo-web-c

Imagina una tienda virtual que tiene todas sus funciones dentro de un mismo sistema. Aunque cada función realiza una tarea diferente, todas pertenecen a una misma aplicación.

WordPress es un sistema de gestión de contenidos (CMS, por sus siglas en inglés) de código abierto y gratuito que permite crear, administrar y publicar sitios web de todo tipo sin necesidad de saber programar.

- El núcleo (Core): Es el software base y gratuito de WordPress.org que controla la estructura general del sitio, la base de datos y la seguridad.
- Los temas (Plantillas): Definen la apariencia visual, el diseño, los colores y la distribución de los elementos de la página.
- Los plugins (Complementos): Son extensiones de software que se instalan para añadir funciones específicas (como formularios de contacto, tiendas en línea con WooCommerce o mejoras de posicionamiento SEO) sin tocar el código.

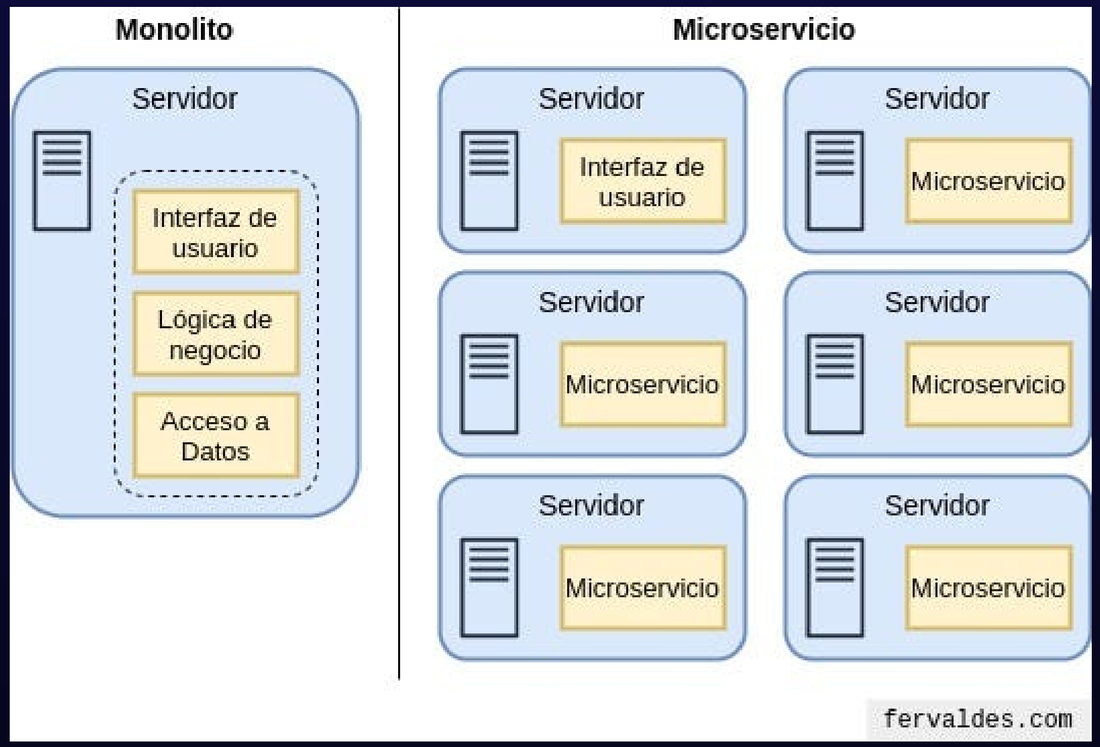
(OpenStax, 2024).

Características principales

- **Unidad de funcionamiento:** Todos los módulos hacen parte de una misma aplicación.
- **Código integrado:** Las funcionalidades comparten una misma base de código o están organizadas dentro de una solución unificada.
- **Dependencia entre módulos:** Los diferentes componentes están relacionados y pueden depender unos de otros.
- **Despliegue conjunto:** Cuando se realiza una actualización, normalmente es necesario desplegar nuevamente la aplicación completa.
- **Administración centralizada:** El sistema se desarrolla, ejecuta y administra como una aplicación unificada.

(IBM, s. f.; OpenStax, 2024).

Figura 9. Principios de diseño y estructuración de sistemas informáticos



https://medium.com/@ktufernando/la-guía-definitiva-de-la-arquitectura-del-software-f419db9c6bf7

Arquitectura Monolitica

¿Cómo se despliega una aplicación monolítica?

El despliegue es el proceso de preparar e instalar una aplicación para que pueda funcionar y ser utilizada por los usuarios.
En una arquitectura monolítica, normalmente se empaqueta la aplicación completa y se publica como una sola unidad.
Existen diferentes formas de realizar este despliegue:

1. **En un servidor:** Se instala y ejecuta toda la aplicación en un servidor físico.
2. **En una máquina virtual:** La aplicación se ejecuta dentro de un entorno virtual que dispone de sus propios recursos.
3. **En contenedores:** La aplicación se empaqueta en un contenedor que incluye lo necesario para ejecutarla.
4. **En la nube:** Se aloja en plataformas que permiten ejecutar y administrar la aplicación desde internet.

(Microsoft, s. f.).

DIFERENCIA ENTRE ARQUITECTURA Y DESPLIEGUE

ARQUITECTURA	DESPLIEGUE
Describe cómo está organizada internamente la aplicación.	Describe dónde y cómo se ejecuta la aplicación.
Define los componentes y sus relaciones.	Define los servidores, máquinas virtuales o contenedores utilizados.
Se enfoca en la estructura del software.	Se enfoca en la ejecución del software.

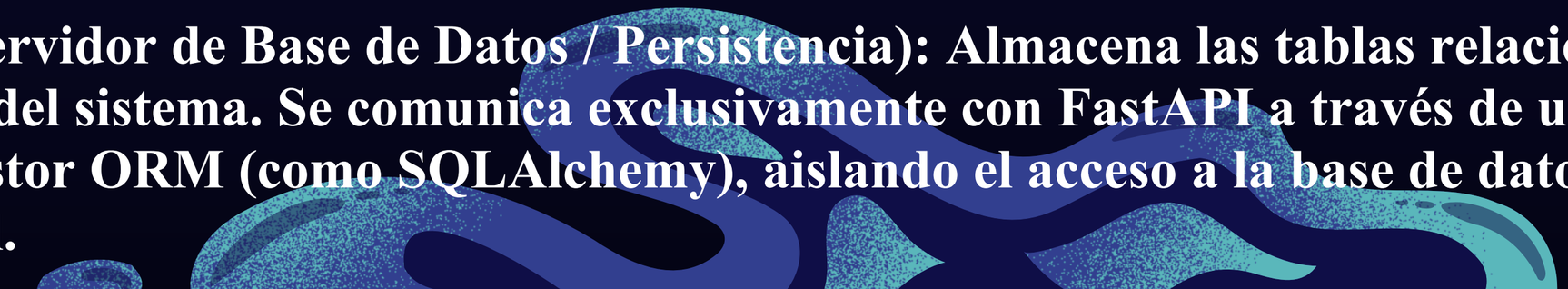
Aplicación al proyecto ADSO

En el desarrollo de proyectos de software en el programa ADSO, el modelo cliente-servidor se aplica estructurando una arquitectura de tres capas (3-Tier):

- React (Cliente / Front-end): Se encarga de la capa de presentación en el navegador. Captura los eventos del usuario, renderiza componentes interactivos y envía peticiones HTTP asíncronas (fetch o axios) empaquetadas en JSON hacia el backend.

- FastAPI (Servidor de Aplicación / Back-end): Funciona como la capa intermedia o API RESTful. Escucha las peticiones en endpoints específicos (/api/v1/...), valida datos de entrada mediante modelos Pydantic, procesa autenticación (JWT) y ejecuta las reglas de negocio.

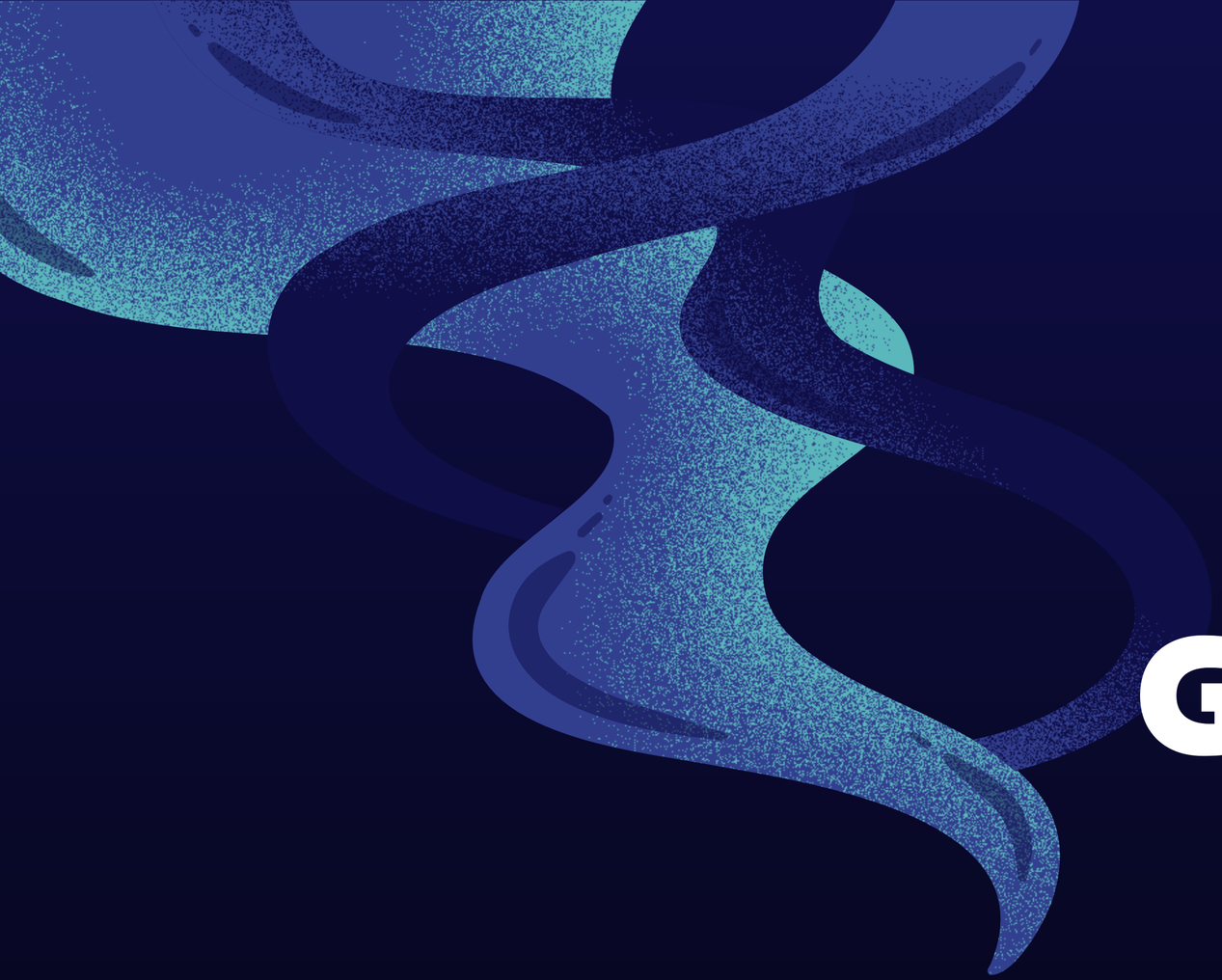
- MySQL (Servidor de Base de Datos / Persistencia): Almacena las tablas relacionales y registros del sistema. Se comunica exclusivamente con FastAPI a través de un driver o gestor ORM (como SQLAlchemy), aislando el acceso a la base de datos del cliente final.



Aspecto	Ventajas	Desventajas y Limitaciones
Seguridad	Las reglas de negocio y las credenciales de la base de datos no quedan expuestas en el cliente.	Vulnerabilidad ante ataques de denegación de servicio (DDoS) o saturación de tráfico.
Mantenimiento	Modificaciones en FastAPI o MySQL surten efecto inmediato sin reconfigurar cada cliente.	Dependencia total del servidor; si el backend cae, la aplicación cliente queda inoperativa.
Escalabilidad	Permite escalar horizontalmente (añadiendo instancias del servidor) o verticalmente (hardware).	Latencia de red inherente durante el transporte de paquetes entre el cliente y el servidor.
Control	Centralización de políticas de respaldo, logs de auditoría y monitoreo.	Mayor complejidad de infraestructura, despliegue y gestión de conexiones concurrentes.

Conclusion

La arquitectura cliente-servidor permite la comunicación y el intercambio de información entre diferentes sistemas, mientras que la arquitectura monolítica integra las funcionalidades de una aplicación en una sola unidad. Ambas pueden complementarse para desarrollar sistemas organizados, seguros y eficientes, dependiendo de las necesidades del proyecto.



Gracias por su atencion

Referencias

Arsys. (s.f.). Todo sobre la arquitectura cliente-servidor. Blog de Arsys.
https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor

Cloud Native Computing Foundation. (2024, 27 de marzo). Monolithic apps.
https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor

Daemon4. (s.f.). Qué es la arquitectura cliente servidor y cómo funciona.
https://www.daemon4.com/empresa/noticias/arquitectura-cliente-servidor/

Supermicro. (s.f.). ¿Qué es la arquitectura cliente-servidor?.
https://www.supermicro.com/es-es/glossary/client-server-architecture

Microsoft. (s. f.). Common web application architectures. Microsoft Learn.
https://www.supermicro.com/es-es/glossary/client-server-architecture

OpenStax. (2024). 12.1 Introduction to cloud-native applications. Introduction to Computer Science.
https://openstax.org/books/introduction-computer-science/pages/12-1-introduction-to-cloud-native-applications

Powell, P., & Smalley, I. (s. f.). What is monolithic architecture? IBM.
https://www.ibm.com/think/topics/monolithic-architecture